Using the Lunar-HFE Code

A developer & user guidebook

How the package fits together, how to run it,
and how to change it without breaking anything

Companion to the science guidebook (`paper/primer/guidebook.tex`)
and the quick map (`docs/CODE_MANUAL.md`). Repository: github.com/rp3gregorio/Lunar-HFE

R. P. Gregorio · Kasai Laboratory, Institute of Science Tokyo

Contents

Chapter 1

Orientation

This guidebook teaches you the *code*. If you want the physics and statistics, read `paper/primer/guidebook.tex`; if you want a one-page map, read `docs/CODE_MANUAL.md`. Here we go deeper: every module, every entry point, the exact function signatures, and a set of worked recipes for the changes you are most likely to make.

1.1 What the code does, in one sentence

Given the Apollo 15 and 17 heat-flow temperature records, the code solves a 1-D heat-conduction model of the regolith to a certified periodic steady state, sweeps the deep thermal conductivity K_d until the modelled meter-scale temperatures best match the data, and quantifies the uncertainty by bootstrap and Bayesian cross-check — then draws every figure and writes every number used in the paper.

1.2 The mental model: three layers

The single most useful thing to internalise is that the repository is **layered**, and imports only ever point **downward** (Figure 1.1).

In plain words

Think of `lunar/` as a *library* you installed (like NumPy), and `scripts/` as the little programs you wrote that *use* that library. You change behaviour by editing the library or its config; you *run* things by calling the scripts.

1.3 How a result travels

Every number in the paper follows the same path (Figure 1.2): configuration → material models → solver → `run_with()` → sweep & bootstrap → a JSON file in `output/` → a figure script → a PDF in `paper/`.

Tip

If you remember only one function name, remember `run_with()`. Once you understand its inputs and its single output, you understand the whole retrieval.

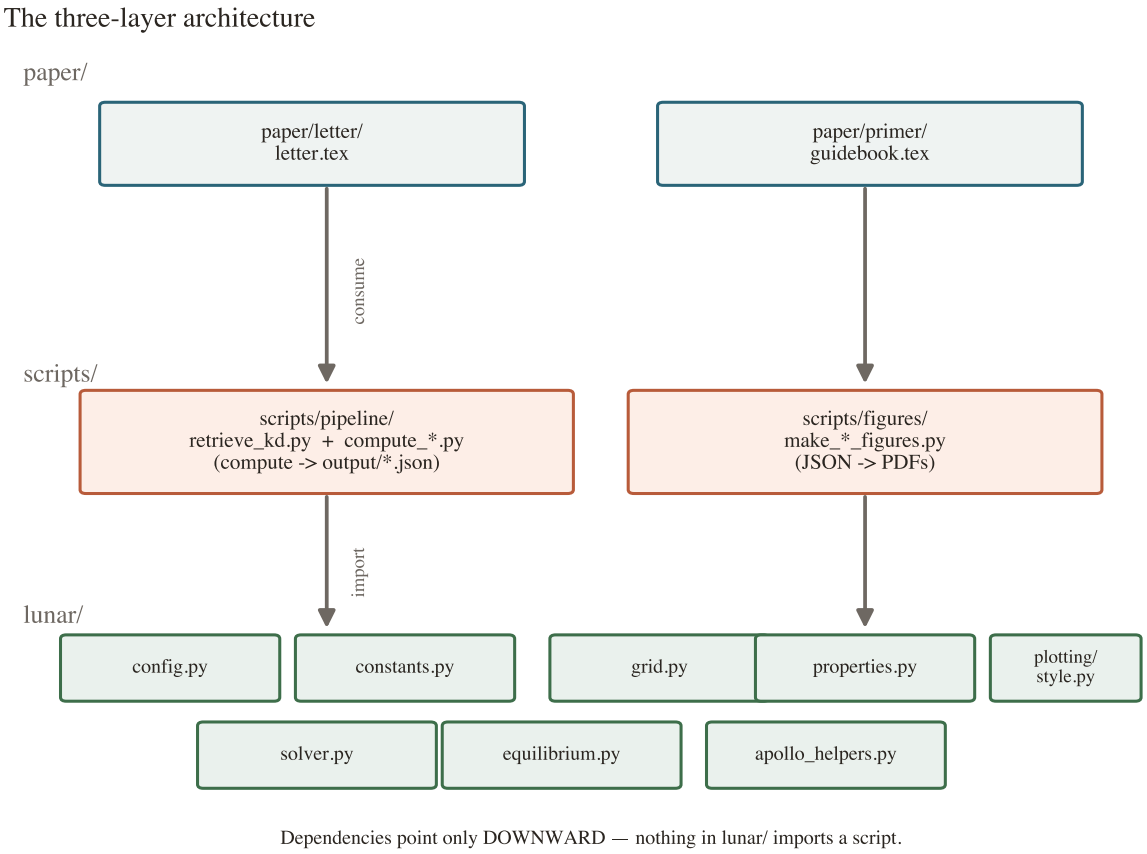

Figure 1.1: The three layers. `lunar/` is the installable package (the physics engine, the one configuration file, and the shared plotting style). `scripts/` are thin command-line drivers that import the package. `paper/` consumes the results. Nothing in `lunar/` imports a script.

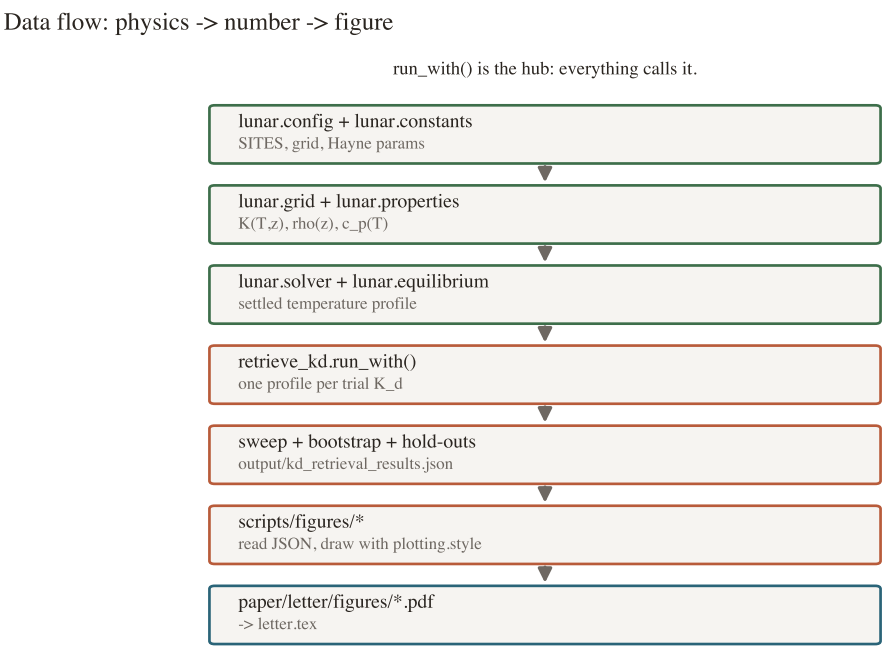

Figure 1.2: The physics→number→figure pipeline. The function `run_with()` in `retrieve_kd.py` is the hub: it turns one trial K_d into one settled temperature profile, and everything above it (sweeps, bootstrap, sensitivities, figures) calls it.

Chapter 2

Setup and your first run

2.1 Prerequisites

- Python $\geq$ 3.10, a C compiler for NumPy/SciPy wheels (already present on macOS/Linux), and `git`.
- A LaTeX distribution (TeX Live / MacTeX) only if you want to compile the PDFs; the science runs without it.
- $\sim$1 GB free disk if you fetch the Diviner cache (optional; only the surface-closure cross-check needs it).

2.2 Install

```
git clone https://github.com/rp3gregorio/Lunar-HFE.git
cd Lunar-HFE
python3 -m venv .venv && source .venv/bin/activate
make install # editable install of the lunar package + dev deps
```

`make install` runs `pip install -e .` so that `import lunar` works from anywhere and your edits to `lunar/` take effect immediately, with no reinstall.

2.3 The one-word entry points

Everything is driven by the `Makefile`. Run `make help` to list targets; the important ones:

`make install`	editable install of `lunar/` + dev deps
`make retrieve`	core K_d retrieval + bootstrap $\rightarrow$ `output/kd_retrieval_results.json`
`make aux`	every sensitivity sweep, model selection, MCMC, Diviner closure
`make figures`	regenerate every figure (calls `scripts/make_all_figures.py`)
`make paper`	compile the letter + guidebook PDFs
`make test`	the `pytest` suite (37 tests)
`make all`	`retrieve` $\rightarrow$ `aux` $\rightarrow$ `figures` $\rightarrow$ `paper`
`make clean`	remove LaTeX build artefacts

2.4 Your first run

```
make retrieve # ~1-2 min: writes output/kd_retrieval_results.json
make test # ~15 s: should report "37 passed"
make figures # redraws the PDFs from the JSON you just made
```

After `make retrieve` you should see, near the end of the log, the two headline numbers

$$K^*_{d,\mathrm{A15}} \approx 4.6, \qquad K^*_{d,\mathrm{A17}} \approx 8.1 \ \ (\mathrm{mW\,m^{-1}\,K^{-1}}),$$

and the file `output/kd_retrieval_results.json` on disk. That file is the canonical result; everything downstream reads it.

Gotcha

The Diviner surface-closure cross-check (`compute_diviner_closure`) needs the GCP cache, fetched once with `python scripts/fetch_diviner.py` (∼310 MB). If you skip it, that single figure is skipped — the retrieval itself does *not* need Diviner.

Chapter 3

The configuration: `lunar/config.py`

This is the **single source of truth**. Almost every change you will ever want starts here, and because every script reads these same objects, a change propagates everywhere at once.

3.1 The site table `SITES`

A dictionary keyed `'A15'` / `'A17'`. Each entry carries the physical description of one borehole:

Key	Example (A15)	Meaning
`tag`	`'A15'`	short site label used in caches
`lat,lon`	26.13, 3.63	landing-site coordinates [deg]
`albedo`	0.131	bolometric Bond albedo
`emissivity`	0.95	IR emissivity
`Q_BASAL`	0.021	basal heat flux [$\mathrm{W\,m^{-2}}$] (held fixed)
`T_MEAN_EFF`	250.0	seed mean temperature [K] (*seed only*)
`MIN_DEPTH_CM`	80	borestem cut: ignore sensors above this
`mission`	`'a15'`	key into the bundled HFE data

Gotcha

`T_MEAN_EFF` only *seeds* the first iterate. The converged profile is independent of it — that guess-independence is exactly the property the equilibrium solver guarantees (audit flag F1). Do not treat it as a tuning knob.

3.2 The other configuration objects

`GRID`	`dict(z_max, dz0, growth)` — depth mesh: max depth, top-layer thickness, geometric growth rate.
`HAYNE`	`dict(K_S, H, CHI, T_REF)` — the Hayne (2017) $K(T, z)$ parameters.
`S0`	solar constant at 1 AU [$\mathrm{W\,m^{-2}}$].
`T_LUNAR`	one lunation [s].
`DT_STEP`	solver time step [s] (3600 s = 1 h).
`EQ_Z_ANCHOR`, `EQ_N_INNER`, `EQ_MAX_OUTER`, `EQ_ANCHOR_TOL`	equilibrium-solver controls (anchor depth, inner cycles, outer cap, convergence tol).
`KD_GRIDS`	per-site array of trial K_d values the sweep walks.
`DEPTH_SIGMA_CM`	sensor-depth jitter ($\pm$2.5 cm) used by the bootstrap.
`TL_*`	3-layer-model parameters (reference depths, densities).

Tip

Worked change. To test Apollo 15 with a 10% higher basal flux, edit one number:

```
SITES['A15']['Q_BASAL'] = 0.0231 # was 0.021
```

then `make retrieve`. The retrieval, every sensitivity script, and every figure now use the new value — no other file to touch.

Chapter 4

The physics engine (`lunar/`)

These are the modules you import, not run. Signatures below are the real ones in the code.

4.1 `constants.py` — cited numbers

Every physical constant lives here with a source comment: the Stefan–Boltzmann σ, the Hayne surface/deep conductivities (K_S, K_d), the H and χ parameters, regolith densities, and the specific-heat / Martínez coefficients. **Never** hard-code a constant in a script — import it from here.

4.2 `properties.py` — material models

```
conductivity_hayne(T, z, Ks=K_SURFACE, Kd=K_DEEP, H=..., chi=...)
conductivity_martinez(T, z=None, rho=None)
density_hayne(z, rho_s, rho_d, H)
specific_heat(T, model="hayne") # or "biele"
```

`conductivity_hayne` returns the temperature- and depth-dependent conductivity $K(T, z) = K_c(z)\,[1 + \chi(T/T_{\mathrm{ref}})^3]$ where the solid term K_c rises from K_S at the surface to K_d at depth. These are pure functions of NumPy arrays — no global state.

4.3 `grid.py` — the depth mesh

```
g = make_geometric_grid(z_max=3.0, dz0=0.002, growth=0.15)
g.z_face # cell-face depths [m], shape (N+1,), z_face[0] == 0
g.z_mid # cell-centre depths [m], shape (N,)
g.dz # cell thicknesses [m]
g.n_layers # number of cells
```

The grid is *geometric*: thin at the top (millimetre cells to resolve the diurnal skin depth) and growing with depth. Uniform grids are forbidden by project rule — they cannot resolve the skin and the meter scale at once.

4.4 `solver.py` — one forward integration

The low-level engine: `solve_pixel(PixelInputs(...))` integrates the Crank–Nicolson heat equation over the supplied forcing with a Newton surface energy balance and a geothermal-flux bottom boundary, returning a `PixelOutputs`. You rarely call this directly — the equilibrium solver wraps it.

4.5 `equilibrium.py` — the settled state (the F1 fix)

This is the function that made the study correct. Its job: return the *periodic steady state*, independent of the initial guess.

```
solve_periodic_equilibrium(
   grid, t, insolation, albedo, emissivity, Q_b, K_func,
   cp_func=None, rho_func=None,
   T_guess=250.0, z_anchor=0.55,
   n_inner=12, max_outer=20, anchor_tol_K=0.005,
) →EquilibriumResult
```

It alternates short skin-depth spin-ups with a mean-field reconstruction that forces the cycle-mean flux to equal Q_b at every depth, iterating until the anchor temperature stops moving. The returned `EquilibriumResult` carries both the answer and its certificate:

`out`	the final periodic cycle (`PixelOutputs`)
`T_mean`	cycle-mean temperature profile [K] — the science output
`n_outer`	outer iterations used
`anchor_K`	final anchor temperature $\langle T \rangle(z_0)$ [K]
`anchor_drift_K`	last inter-iteration change (convergence proof)
`flux_closure`	$\max \lvert\langle q \rangle - Q_b\rvert / Q_b$ below the anchor
`converged`	boolean

In plain words

"Periodic steady state" means: run the model long enough that each lunation looks like the last one, and deep down the heat flowing through matches the geothermal flux. `anchor_drift_K` and `flux_closure` are the two numbers that *prove* you got there.

4.6 A minimal forward run by hand

You can drive the engine yourself without any of the retrieval machinery:

```
import numpy as np
from lunar.config import SITES, GRID, HAYNE, S0, T_LUNAR, DT_STEP
from lunar.grid import make_geometric_grid
from lunar.properties import conductivity_hayne, specific_heat
from lunar.equilibrium import solve_periodic_equilibrium

site = SITES['A15']
g = make_geometric_grid(**GRID)
t = np.arange(0, T_LUNAR, DT_STEP)
cosz = np.clip(np.cos(2*np.pi*t/T_LUNAR), 0, None) # toy forcing
insol = S0 * (1 - site['albedo']) * cosz

def K(T, z):
   return conductivity_hayne(T, z, Ks=HAYNE['K_S'],
                        Kd=4.6e-3, H=HAYNE['H'],
                        chi=HAYNE['CHI'])
eq = solve_periodic_equilibrium(
      grid=g, t=t, insolation=insol,
      albedo=site['albedo'], emissivity=site['emissivity'],
      Q_b=site['Q_BASAL'], K_func=K,
      cp_func=lambda T: specific_heat(T, model='hayne'),
      T_guess=site['T_MEAN_EFF'])
print(eq.converged, eq.anchor_drift_K, eq.T_mean[-1])
```

Chapter 5

The retrieval pipeline (`scripts/pipeline/`)

5.1 `run_with()` — the hub

```
run_with(site_cfg, *, kd=None, h=None, qb=None,
         k_model='hayne', rho_deep=None, martinez_alpha=None)
   →(z_mid, T_mean)
```

Give it a site config and a trial K_d; it builds the grid and the forcing, assembles the chosen conductivity model (`'hayne'`, `'3layer'` or `'martinez'`), calls `solve_periodic_equilibrium`, caches the result, and returns the mid-cell depths and the settled mean profile. Results are memoised, so repeated calls with the same arguments are free.

`kd`	deep conductivity [$\mathrm{W\,m^{-1}\,K^{-1}}$]; required for `hayne`/`3layer`
`h`	override the Hayne H depth scale (else config)
`qb`	override the basal flux (else `site['Q_BASAL']`)
`k_model`	`'hayne'` (default), `'3layer'`, or `'martinez'`
`rho_deep`	deep bulk density override (3-layer model)
`martinez_alpha`	density scalar for the Martínez per-site retrieval

5.2 From profile to K_d^*

`run_with` is called once per trial K_d to build a residual matrix R (modelled − observed at each deep sensor). Then:

```
kd_star, rmse_star = kd_star_from_residuals(R, kd_grid)
```

fits a parabola to the RMSE-vs-K_d curve and returns the sub-grid minimum. Re-sampling sensors (with the $\pm 2.5\,\mathrm{cm}$ depth jitter) and repeating $N_{\mathrm{boot}} = 1500$ times gives the bootstrap confidence interval. The hold-out checks (train on one sensor set, predict the other; leave-one-deepest-out) reuse the same residuals.

5.3 What `retrieve_kd.py` writes

Running it (`make retrieve`) produces `output/kd_retrieval_results.json` plus the bootstrap, robustness and hold-out figures. That JSON is the spine of the paper.

5.4 The `compute_*` sensitivity scripts

Each is a thin driver that imports `run_with` / `kd_star_from_residuals` from `retrieve_kd.py` and writes one JSON. Because they all import the engine through `retrieve_kd.py` (which re-exports the single `lunar.config.SITES`), they cannot drift out of sync with the main retrieval.

Script	Writes
`compute_headline_rmse.py`	`headline_rmse.json`
`compute_borestem_sensitivity.py`	`borestem_sensitivity.json`
`compute_stability_threshold_sensitivity.py`	`stability_threshold_sensitivity.json`
`compute_surface_bias_test.py`	`surface_bias_test.json`
`compute_uniform_kd_sensitivity.py`	`uniform_kd_test.json`
`compute_fixed_input_sensitivities.py`	`fixed_input_sensitivities.json`
`compute_model_selection.py`	`model_selection.json` (AICc M1/M2/M3)
`compute_error_budget.py`	`kd_error_budget.json` (reads the above)
`bayesian_crosscheck.py`	`bayesian_crosscheck_samples.json` + posterior figs
`compute_diviner_closure.py`	`diviner_closure.json` + closure fig

`make aux` runs all of them in the right order (`error_budget` last, since it consumes the others).

Chapter 6

The results: `output/*.json`

The JSON files are the contract between the pipeline and the figures: the pipeline writes them, the figure scripts and the paper read them. The canonical one, `kd_retrieval_results.json`, contains for each site:

- `kd_star`, `rmse_star` — the retrieved conductivity and its RMSE;
- `kd_grid`, `rmse_curve` — the full sweep (for the K_d-sweep figure);
- `bootstrap` — `median`, `ci_lo`, `ci_hi`, and the raw `samples`;
- `holdout`, `loo_deepest` — the cross-prediction diagnostics;
- provenance (git hash, timestamp, config echo) so a figure can never be drawn from stale numbers silently.

Tip

To read a number out without any code:

```
python -c "import json;d=json.load(open('output/kd_retrieval_results.json'));print(d
    ['A17']['kd_star'])"
```

Chapter 7

The figures (scripts/figures/)

Figure scripts *read* output/*.json and *draw* with the shared style; they never recompute science. Every one imports lunar.plotting.style (palette + rcParams) and lunar.config (for SITES); the few that need a forward curve import run_with.

Script	Produces
make_intro_figures.py	probe schematic
make_context_map_figure.py	landing-site map
make_apollo_timeline.py	stability-window timeline
make_letter_figures.py	amplitude, mean-T, K_d sweep
make_results_figures.py	bootstrap + robustness panels
make_alpha_sweep_figure.py	Martínez α-sweep
make_book_figures.py, make_primer_figures.py	guidebook teaching figures
make_equilibrium_certification.py	solver certification (to output/figures/)
make_codeguide_figures.py	the two diagrams in *this* manual

scripts/make_all_figures.py (make figures) runs them all, timing each and reporting failures without stopping.

7.1 The shared style

lunar/plotting/style.py exports the palette (C_A15, C_A17, C_HAYNE, ...), figure-width constants, and helpers fmt_axis, legend_below, legend_outside. Use them so every figure matches:

```
from lunar.plotting.style import C_A15, C_A17, fmt_axis, legend_below
fig, ax = plt.subplots()
ax.plot(x, y, color=C_A15, label="Apollo 15")
fmt_axis(ax, xlabel="depth [m]", ylabel="T [K]")
legend_below(ax)
```

Gotcha

Site *presentation* colours live in the style module, not in config.SITES. The config holds only physics. When a figure needs a per-site colour, map it: SITE_COLOR = {"A15": C_A15, "A17": C_A17}.

Chapter 8

Recipes

Change a site parameter

Edit the one number in `SITES` (Chapter 3) and `make retrieve`.

Retrieve under the Martínez model instead of Hayne

Call `run_with(site, k_model='martinez', martinez_alpha=a)` in a sweep over `a`; this is exactly what `make_alpha_sweep_figure` does. No engine change needed.

Run a single site interactively

```
import sys; sys.path.insert(0, "scripts/pipeline")
from retrieve_kd import run_with
from lunar.config import SITES
z, T = run_with(SITES['A17'], kd=8.1e-3)
```

Regenerate just one figure

```
python -c "import sys;sys.path.insert(0,'scripts/figures');import make_letter_figures as
     m;m.main()"
```

Add a new site

Add an entry to `SITES` with the eight fields, add a `KD_GRIDS['NEW']` array, ensure the bundled HFE data has the matching `mission` key, and the entire pipeline picks it up. Add a colour in `plotting/style.py` if you will plot it.

Add a new conductivity model

Write the pure function in `properties.py`, register a branch for it in `run_with`'s `k_model` dispatch, and it becomes available to every sweep and figure.

Add a new sensitivity test

Copy the smallest `compute_*.py`, import `run_with` / `kd_star_from_residuals`, write your JSON, and add a line to `Makefile`'s `aux` target.

Chapter 9

Testing and reproducibility

`make test` runs the `pytest` suite (37 tests): grid monotonicity and skin resolution, conductivity/density model values against cited numbers, solver energy conservation and the analytic thermal-wave check, equilibrium guess-independence (start from two different `T_guess` values and assert the same converged profile), and JSON-schema checks on the committed results.

- **Determinism:** the bootstrap and MCMC seed their RNGs, so reruns reproduce the published intervals bit-for-bit.
- **Provenance:** each JSON records the git hash and config it was made with; a figure drawn from a stale file is detectable.
- **Audit trail:** `docs/FLAG_REPORT.md` lists every issue found in review (F1–F6) and how it was resolved.
- **Notebooks:** the five `notebooks/` walk the same pipeline interactively and are a good place to learn by stepping.

Chapter 10

Toward the C++ port

The Python layout is deliberately a clean reference for a faster C++ core. The numerics are isolated in three small, dependency-light modules — `grid.py`, `solver.py`, `equilibrium.py` — with no plotting or file I/O mixed in, all fed by a single `config.py`.

1. Port `grid.py` (pure array construction) first — trivial and fully testable against the Python output.
2. Port `solver.py` (Crank–Nicolson + Thomas solve + Newton surface balance). Validate against the analytic thermal wave the Python test already uses.
3. Port `equilibrium.py` (the outer flux-anchored iteration).
4. Keep `config.py` as the single parameter file the C++ side *also* reads (emit it as JSON), so there is still one source of truth.
5. Leave the analysis (`scripts/pipeline/`) and all figures in Python, calling the fast core through a thin binding.

Tip

Port bottom-up and test each layer against the Python reference before moving up. The 37-test suite is your regression harness across languages.

Chapter A

Command cheat-sheet

```
make install # editable install
make retrieve # core retrieval + bootstrap -> output/kd_retrieval_results.json
make aux # all sensitivity sweeps + MCMC + closure
make figures # redraw every figure
make paper # compile the PDFs
make test # pytest (37 tests)
make all # everything, from scratch
make clean # remove LaTeX build artefacts

# standalone equivalents
python scripts/pipeline/retrieve_kd.py
python scripts/make_all_figures.py
python scripts/figures/make_codeguide_figures.py # this manual's diagrams
pytest -q
```

Chapter B

Glossary of key objects

`SITES`	the site table; one physics record per borehole
`make_geometric_grid`	builds the thin-at-top depth mesh (`DepthGrid`)
`conductivity_hayne`	$K(T, z)$ used in the baseline retrieval
`solve_periodic_equilibrium`	the guess-independent settled-state solver
`EquilibriumResult`	the converged profile + its convergence certificate
`run_with`	the hub: one trial $K_d \to$ one settled profile
`kd_star_from_residuals`	parabola-fit sub-grid minimum of the RMSE curve
`kd_retrieval_results.json`	the canonical result file everything downstream reads

Generated as a companion to the Lunar-HFE repository. For the science behind the code, see `paper/primer/guidebook.tex`.